

Logistic Regression for Imbalanced Dataset

```
In [12]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
In [13]: from collections import Counter
from sklearn.datasets import make_classification
```

```
In [14]: #imbalanced dataset
X,y=make_classification(n_samples=10000,n_features=2,n_clusters_per_class
```

```
In [15]: X
```

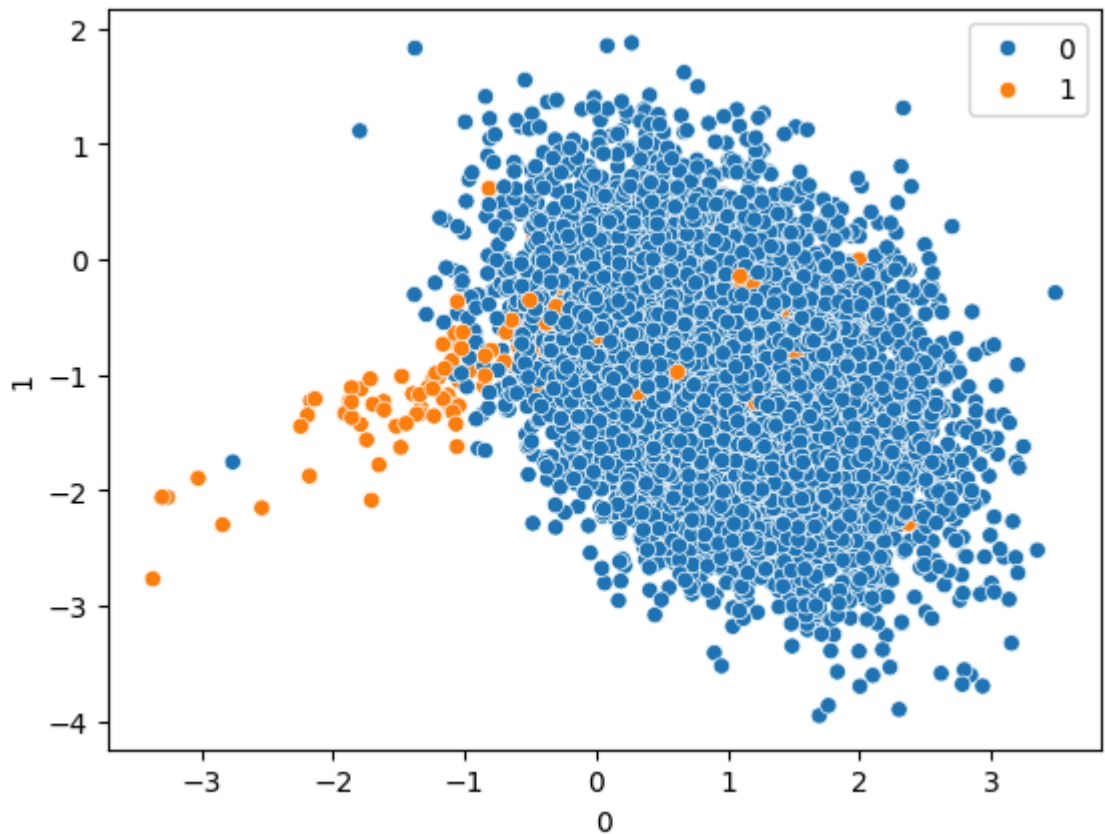
```
Out[15]: array([[ 1.87991016, -1.4850347 ],
 [ 0.37708311, -2.23858488],
 [ 1.35431163, -2.66463822],
 ...,
 [ 0.99509904, -0.54192355],
 [ 1.75859359, -1.54961188],
 [ 0.79770543, -1.99467372]], shape=(10000, 2))
```

```
In [16]: Counter(y)
```

```
Out[16]: Counter({np.int64(0): 9846, np.int64(1): 154})
```

```
In [17]: sns.scatterplot(x=pd.DataFrame(X)[0],y=pd.DataFrame(X)[1],hue=y)
```

```
Out[17]: <Axes: xlabel='0', ylabel='1'>
```



```
In [18]: #split the dataset into train amd test
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.25,random_
```

```
In [19]: ##hyperparameter tuning
from sklearn.linear_model import LogisticRegression
model=LogisticRegression()
penalty=['l1','l2','elastic']
c_values=[100,10,1.0,0.1,0.01]
solver=['newton_cg','lbfgs','liblinear','sag','saga']
class_weight=[{0:w,1:y} for w in [1,10,50,100] for y in [1,10,50,100]]
```

```
In [20]: params=dict(penalty=penalty,c=c_values,Solver=solver,class_weight=class_w
```

```
In [21]: params
```

```
Out[21]: {'penalty': ['l1', 'l2', 'elastic'],
          'c': [100, 10, 1.0, 0.1, 0.01],
          'Solver': ['newton_cg', 'lbfgs', 'liblinear', 'sag', 'saga'],
          'class_weight': [{0: 1, 1: 1},
                           {0: 1, 1: 10},
                           {0: 1, 1: 50},
                           {0: 1, 1: 100},
                           {0: 10, 1: 1},
                           {0: 10, 1: 10},
                           {0: 10, 1: 50},
                           {0: 10, 1: 100},
                           {0: 50, 1: 1},
                           {0: 50, 1: 10},
                           {0: 50, 1: 50},
                           {0: 50, 1: 100},
                           {0: 100, 1: 1},
                           {0: 100, 1: 10},
                           {0: 100, 1: 50},
                           {0: 100, 1: 100}]}
```

```
In [25]: from sklearn.linear_model import LogisticRegression
         from sklearn.model_selection import StratifiedKFold, GridSearchCV

         logistic = LogisticRegression()
         params = {
             'C': [100, 10, 1.0, 0.1, 0.01],
             'penalty': ['l2'],
             'solver': ['lbfgs']
         }

         cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

         grid = GridSearchCV(
             estimator=model,
             param_grid=params,
             cv=cv,
             n_jobs=1
         )

         grid.fit(X_train, y_train)
         logistic.fit(X_train, y_train)
         y_pred = logistic.predict(X_test)
```

```
= 'l1', and C=np.inf instead of penalty=None.  
warnings.warn(
```

```
In [27]: grid.best_params_
```

```
Out[27]: {'C': 100, 'penalty': 'l2', 'solver': 'lbfgs'}
```

```
In [28]: y_pred=grid.predict(X_test)
```

```
In [31]: #GridSearchCV  
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report  
  
score=accuracy_score(y_test,y_pred)  
print(score)  
cm=confusion_matrix(y_test,y_pred)  
print(cm)  
print(classification_report(y_test,y_pred))
```

```
0.9888
```

```
[[2469  1]  
 [  27  3]]
```

		precision	recall	f1-score	support
	0	0.99	1.00	0.99	2470
	1	0.75	0.10	0.18	30
	accuracy			0.99	2500
	macro avg	0.87	0.55	0.59	2500
	weighted avg	0.99	0.99	0.98	2500

```
In [ ]:
```